

Il gruppo di lavoro per la realizzazione di questo progetto è formato da: Christian Torli, Tommaso Roncato e Alvis Bert.

A seguire uno schema delle principali funzionalità implementate con conseguenti difficoltà che abbiamo incontrato. Accanto alla funzionalità sarà inoltre presente il nome della persona che più si è occupata della sua gestione:

- **Inizializzazione progetto:** cartelle, repository e makefile [Christian]: abbiamo diviso il progetto in due cartelle: src e include in maniera tale da dividere file .c e .h, sono state trovate lievi difficoltà nella corretta inizializzazione di cmake ma nulla di rilevante.
- **Creazione di utils.c:** [Christian] riscontrate alcune difficoltà nella realizzazione della funzione clear_screen() che ha il compito di pulire il terminale. La soluzione trovata più appropriata al problema è stata l'uso di una ANSI escape sequence invece di system call. È la soluzione più portabile e ormai quasi tutti i terminali le supportano.
- **Gestione dei menù:** [Christian] l'obiettivo era di avere un sistema che potesse rappresentare tutti i menù di gioco, fosse accessibile facilmente e semplice da lavorare. La soluzione è stata la funzione variadica select_option() che ha come parametri il prompt da chiedere all'utente e un qualsiasi numero di altre stringhe che sarebbero diventate le possibili scelte; la funzione si sarebbe inoltre occupata di filtrare gli input non validi e ritornare l'input dell'utente.
- **Serializzazione dei dati:** [Christian] è stato deciso di rappresentare i salvataggi come file di testo, ognuno formattato come "savingX" dove X è il numero del salvataggio. Per salvare la partita è sufficiente aggiungere un file alla cartella savings con l'index pari a 1+ rispetto al numero dell'ultimo salvataggio. L'obiettivo era di realizzare un sistema che potesse individuare l'ultimo file di salvataggio e mostrarlo per primo nella lista e a seguire i salvataggi successivi. Ci siamo quindi ritrovati davanti a più problemi: innanzitutto se venisse eliminato un file, la corrispondenza tra numero dei file e loro posizione non sarebbe più rispettata, poi era necessario individuare il file con il numero più alto e infine serviva di contare quanti file erano presenti nella cartella. Sono quindi state realizzate alcune funzioni di supporto come get_last_index() e get_nth_index() (per dettagli vedere documentazione su savings.h).

Arrivati a questo punto, la gestione dei file di salvataggio funzionava correttamente ma con un numero fisso di salvataggi massimi. Per eliminare questa limitazione è stata necessaria la funzione count_files() che utilizza la libreria dirent.h per scorrere la cartella savings e inizializzare l'array contenente i dati dei salvataggi di conseguenza. Aver introdotto questa libreria ha tuttavia richiesto l'uso di conditional import. I sistemi muniti della libreria a livello di sistema (UNIX) avrebbero utilizzato quella di sistema, su windows invece sarebbe stata utilizzato il sorgente di dirent compilato appositamente.

- **Rappresentazione ed esplorazione dungeon** [Tutti]: È stato deciso di creare una struct chiamata entity e un array per livello contenente tutte le struct inizializzate, una per nemico.
È stato poi creato un array contenente i 3 livelli.
Accedere ad una specifica entity avrebbe quindi voluto dire accedere all' Array dei livelli alla posizione dell' indice della missione interessata e all' indice del nemico interessato.
Ottenere quindi nemici casuali era quindi solo questione di accedere alla `level_table[LEVEL_ID][roll_dice()-1]`
- **La gestione del combattimento** [Alvise, Tommaso] è stata quindi questione di smistare diverse casistiche, stampare le stringhe corrette e mettere assieme pezzi.
- **Gestione di inventario e negozio** [Christian, Tommaso]: una volta codificati gli oggetti, applicato il loro effetto in battaglia e realizzato il negozio ci siamo scontrati nel problema legato alla loro memorizzazione (sistema scelto anche per le missioni poi). Era infatti necessario memorizzare 5 oggetti il cui possesso era esprimibile con un valore booleano e le porzioni cumulabili, tutto con un singolo valore intero.
Questo problema è stato risolto mediante l' impiego di divisioni e moduli per potenze di 10 (vedi documentazione su item.c per maggiori dettagli)
- **Completamento missioni e il loro salvataggio** [Christian, Alvise]: avendo già scelto di adottare lo stesso sistema di salvataggio degli item, questo compito è stato pressoché immediato.
- **Drago e Signore Oscuro** [Alvise]: la sequenza di Padovan mediante un algoritmo ricorsivo e il gioco di scudo magia spada sono quindi state realizzate con successo, per la gestione del combattimento finale.
- **Trucchi** [Christian]: l'implementazione del codice KONAMI è stata estremamente facile grazie alla già presente infrastruttura, la difficoltà è stata più legata allo sblocco della missione finale mediante trucchi, cosa che in origine non era stata preventivata è stato quindi necessario revisionare il menù di selezione delle missioni per includere quella possibilità.

Per tutta la durata del lavoro, il progetto ha seguito uno standard rigido e fisso ed è stato documentato accuratamente.

È stata inoltre effettuata la scelta ponderata [Christian]: di tenere l'intero software sullo stack. Non è infatti presente nemmeno una malloc. Scelta deliberata in quanto non ritenuta necessaria. Riteniamo che se qualcosa può essere fatto senza uso dello heap e con conseguenze negative a livello di performance o pulizia del codice trascurabili, debba essere fatto senza. Ciò aiuta a prevenire errori quando il software cresce in dimensione data la facilità con la quale si può incappare in use after free, o double free.

Tirando le somme, la realizzazione di questo software è stata fluida e non abbiamo riscontrato rilevanti problemi se non legati alla gestione del lavoro in gruppo e nelle tempistiche.

Compilare il progetto:

```
gcc src/*.c -linclude -lexternal/dirent -std=c99 -o dungeon_crawler  
.\dungeon_crawler
```